

Search & Pathfinding Visualizer

1. Introduction

In this project, the idea behind combining algorithmic problem-solving with actual coding implementation has been utilized. The primary goal of the assignment is to develop an application which finds the optimal path between the initial and destination points in a maze with weights and displays it in a graph format to the user. The method that has been chosen to achieve such results is Dijkstra's shortest path algorithm. It is possible to specify a variety of parameters regarding the starting and ending points as well as other obstacles and regions within the application. Particularly, a user should be able to add walls as well as swamps which will have a higher traversal cost than the regular cells within the graph. Thus, in addition to providing an algorithm for finding the path between two nodes, Dijkstra's algorithm should identify the path with the minimal cost overall. During the course of solving this task, a visual representation of traversed nodes as well as the resulting minimum cost path will appear. In this assignment, each regular cell is represented by the cost of 1, while the cost of cells in swamps is equal to 5. Thus, in order to find the best way of reaching a specified destination, one should solve not only a maze problem, but also optimize the traversal path according to the overall cost of this journey. In other words, the project can be called a weighted path-finding problem, and thus a solution based on Dijkstra's algorithm is relevant here. The project has been implemented utilizing Python language with the usage of Tkinter for GUI development. The implementation of a priority queue needed for executing Dijkstra's algorithm has been realized with the usage of the heapq module.

2.Problem Definition

The basic problem tackled in this project is to identify the optimal path between two points. However, in contrast to the traditional maze-solving problem, the problem at hand features a weighted environment. Specifically, the user specifies the starting point and ending point of their search. Moreover, the user can change the environment through customizing the cells in the grid. The grid structure is set to represent a two-dimensional space where each cell refers to one state. The empty cell refers to the cells that can be traversed freely, while wall cells cannot be traversed. Swamps represent high cost traversable states. In addition, during the algorithmic process, nodes that are already visited are shown in color "visited" and the resulting minimum cost path in "path" color. This way, the user can follow both the progress and results obtained through the algorithm. In fact, the major difference is that not all nodes have the same traversal cost. In particular, traversing an empty cell costs 1 point, while traversing a swamp costs 5 points. As for walls, they cannot be traversed at all. Thus, the task is not merely to get from start point to the end, but rather to find a path to the end state which will have minimum total cost. This approach makes it possible to represent the problem as a graph problem where each state can be regarded as a node and traversing between neighboring cells will correspond to moving along the edge. At the same time, the movement costs will be different from edge to edge which means that the problem is a weighted shortest path problem. Hence, the accumulated cost needs to be taken into account.

3.Why Dijkstra's Algorithm Was Chosen

Dijkstra's algorithm was preferred for use within this project because the project itself is based on the concept of finding a path with weighted cost. All normal cells within this project carry weight of 1, while swamp cells carry weight of 5. Wall cells cannot be passed. Thus, the aim of this particular task is not only finding a path to the destination point, but finding the path which has minimum cost. BFS, DFS and Dijkstra algorithms can be applied to pathfinding tasks. However, in each particular case they provide unique benefits due to their structure. BFS algorithm can be used if the cost for each step along the way is the same and the main aim is to find the path with minimal number of steps. DFS algorithm can be

used to perform searching as well, however, it does not guarantee either minimum cost or minimum number of steps needed to complete the task. The Dijkstra algorithm aims to find a path with minimum cost in positively-weighted structures. Therefore, assigning swamp cells the weight of 5 was also an intentional step since this would enable the system to choose the best among paths which differ in both cost and steps required. Thus, the chosen Dijkstra algorithm is perfectly suited for this particular case structure.

4. Mathematical and Theoretical Background of Dijkstra's Algorithm

4.1 Theoretical Principle of Dijkstra's Algorithm

Dijkstra's algorithm searches for the minimum path from one source node to other nodes in a weighted graph, particularly with non-negative weights. In our case, we treat the grid as a graph where each cell is considered as a node, and movements between neighboring cells are considered as edges since we have only positive costs in our case. At each step of Dijkstra's algorithm, we choose the node with the lowest weight and examine its adjacent nodes; then, we update the cost of an adjacent node if a shorter path exists. This greedy approach is applicable here due to non-negative weights. When we process a node with its minimal cost, no other path to that node can provide better results. Minimum cost information propagates from the source node to all possible destination nodes, thus once we reach the destination, we know the path is valid and the cost is minimal. In the project task, we cannot consider simple counting of cells since their traversal costs may vary. Thus, using Dijkstra's algorithm is the right way to obtain a minimum cost path.

4.2 Mathematical Model and Implementation Logic

The foundation for Dijkstra algorithm can be explained mathematically as a process of determining and incrementally updating the cost of travel from the source node to all other nodes. The approach for this project maintains an incremental cost for every cell and assigns the source cell a cost of zero while assigning infinity to all other cells.

Formula: $\text{dist}(v) = \min(\text{dist}(v), \text{dist}(u) + w(u,v))$

Here, $\text{dist}(u)$ is the current minimum cost to node u , $w(u,v)$ is the traversal cost from u to v , and $\text{dist}(v)$ is the previously known cost for node v . If a lower cost is found, $\text{dist}(v)$ is updated. This update step is known as relaxation in Dijkstra's algorithm.

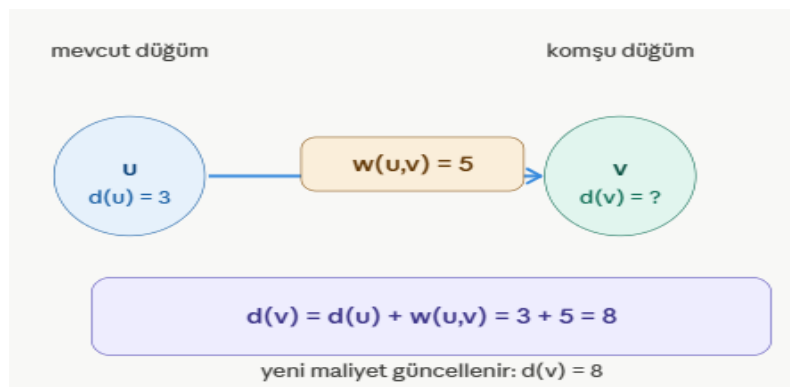


Figure.1

Figure 1. An example of the candidate cost update step in Dijkstra's algorithm for the neighbor transition with weight. Here we see how the candidate cost is computed for a neighboring cell by Dijkstra's algorithm. In reality, this new computed value would be compared to the current cost of the neighbor cell; the smallest value would then be retained. In this fashion, the minimum sum of costs for each node would eventually be obtained. The mathematical concept is used in the following way within the created program. The `dist` construct holds information regarding the minimum cost for each cell, while the `prev`

holds information about the previous nodes in order to find out the path later on. Moreover, a priority queue is created via heapq in order to select the cell with the smallest cost.

```
1 dist = {(r,c): float('inf') for r in range(ROWS) for c in range(COLS)}
2 prev = {}
3 dist[self.start_cell] = 0
4 pq = [(0, self.start_cell)]
```

Figure.2

Inside the loop, the node with the minimum cost is selected from the queue, and then it checks the neighboring nodes. In case a better path is discovered, then that node gets updated, and it is placed back into the queue. The above part illustrates the working principle of the algorithm in programming code.

```
cost, u = heapq.heappop(pq)
for nr, nc, w in self.get_neighbors(u[0], u[1]):
    v = (nr, nc)
    nd = dist[u] + w
    if nd < dist[v]:
        dist[v] = nd
        prev[v] = u
        heapq.heappush(pq, (nd, v))
```

Figure.3

When the algorithm reaches the destination node, the minimum-cost route is reconstructed backward using the prev structure. As a result, the system not only calculates the total cost, but also visualizes the actual path corresponding to that cost. This shows that the theoretical and mathematical logic of Dijkstra's algorithm has been correctly implemented in the developed application.

5. Brief Overview of the Code Architecture

The code revolves around the MazeApp class that holds GUI features, user input functions, and algorithm implementation all together in one object. At first, grid, starting and ending cells, brushes, and other data structures, such as lists of visited nodes, the maze itself, and list of minimum cost nodes, are being initialized. User input functions, based on mouse clicks, follow after this initialization stage and the main algorithmic operations occur during the run() method execution. Auxiliary methods are also included into the code in order to preserve its readability. In terms of architecture, an attempt has been made to make it as interactive as possible. Thus, the code not only finds the shortest path, but also provides visualization of the algorithm work with visited cells, costs of each cell, and finally the shortest way through the maze.

```
class MazeApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Dijkstra - Weighted Maze Visualizer")
        self.root.configure(bg=BLACK)

        self.grid = [["empty"]*COLS for _ in range(ROWS)]
        self.start_cell = None
        self.end_cell = None
        self.mode = "start"
        self.brush = "wall"
        self.running = False
        self.visited = set()
        self.path = set()
```

Figure.4

This section was selected because it shows the core structure of the application. The program is built around the MazeApp class, and this part initializes the main data structures used throughout the project, including the grid, start and end points, interaction mode, and the sets used to store visited nodes and

the final shortest path. This initialization is important because it defines the overall state of the application before the algorithm starts.

```
def on_click(self, event):
    if self.running:
        return
    cell = self.get_cell(event)
    if not cell:
        return
    r, c = cell

    if self.mode == "start":
        self.start_cell = (r,c)
        self.grid[r][c] = "empty"
        self.mode = "end"
        self.info.config(text="2. Click the end cell")
    elif self.mode == "end":
        if (r,c) != self.start_cell:
            self.end_cell = (r,c)
            self.grid[r][c] = "empty"
            self.mode = "draw"
            self.info.config(text="3. Draw terrain, then click Run")
            self.run_btn.config(state=tk.NORMAL)
        else:
            self.info.config(text="Start and end cells must be different")
    elif self.mode == "draw":
        if (r,c) != self.start_cell and (r,c) != self.end_cell:
            (function) draw: Any self.brush
        self.draw()
```

Figure.5

This part was selected because it represents the main user interaction logic of the application. The function controls the step-by-step workflow: first selecting the start cell, then the destination cell, and finally allowing the user to draw terrain types such as walls or swamp cells. This design makes the application interactive and ensures that the pathfinding process starts only after the required inputs are define

```
def get_neighbors(self, r, c):
    result = []
    for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
        nr, nc = r+dr, c+dc
        if 0 <= nr < ROWS and 0 <= nc < COLS:
            t = self.grid[nr][nc]
            if t != "wall":
                result.append((nr, nc, TERRAIN[t]["weight"]))
    return result
```

Figure.6

The above code was chosen as it facilitates the implementation of the Dijkstra algorithm. The function evaluates only movements on the four directions and avoids wall cells. It also calculates the cost of moving to any adjacent cells, an essential aspect of the maze as it is weighted.

```
cost, u = heapq.heappop(pq)
if u in self.visited:
    continue
self.visited.add(u)

if u == self.end_cell:
    found = True
    break

for nr, nc, w in self.get_neighbors(u[0], u[1]):
    v = (nr, nc)
    nd = dist[u] + w
    if nd < dist[v]:
        dist[v] = nd
        prev[v] = u
        heapq.heappush(pq, (nd, v))
```

Figure.7

The following part presents the fundamental idea of Dijkstra's algorithm implemented within the application, which is taking the shortest path node from the priority queue, marking it as visited, and checking all its neighboring nodes. If there is a shorter path to any of those neighbors, then their distances and predecessors are adjusted accordingly.

```
if found:
    node = self.end_cell
    while node in prev:
        self.path.add(node)
        node = prev[node]
    self.path.add(self.start_cell)
    self.draw()
    self.info.config(text=f"Found! Total cost: {dist[self.end_cell]}")
else:
    self.draw()
    self.info.config(text="No path found!")
```

Figure.8

This part explains how the final output is generated and presented. If there is a valid path, the path is reversed using the prev dictionary and stored as self.path. Next, the path is displayed in the interface together with the cost, ensuring that both are accurate and comprehensible.

Demonstration Video: https://youtu.be/_9VBjD6drO4